3D_reconstruction

1. Triangulation
    1. $\vec{x} = P\vec{X}$
    2. $$\begin{bmatrix} x \\ y \\ z \end{bmatrix} = \alpha \begin{bmatrix} p_1 & p_2 & p_3 & p_4 \\ p_5 & p_6 & p_7 & p_8 \\ p_9 & p_{10} & p_{11} & p_{12} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$
    3. How do we solve for unknowns in a similarity relation?
    4. Direct Linear Transform(DLT): remove scale factor, convert to linear system and solve with SVD.
2. Bundle Adjustment
    1. From initial triangulated 3D points, jointly optimizing 3D coordinates and camera parameters by minimizing the reprojection error.
    2. Similar to the non-linear calibration refinement.
    3. Reprojection error
        1. The reprojection error is the Euclidean distance (in pixels) between an observed image point and the corresponding 3D point reprojected onto the camera frame.
        2. The reprojection error gives us a quantitative measure of the accuracy of the calibration (ideally it should be zero).
        3. $\pi(P_W^i, K, R, T)$ Reprojected point, $||p^i - \pi(P_W^i, K, R, T)||^2$
    4. Non-linear calibration refinement
        1. The calibration parameters K, R, T determined by the DLT can be refined by minimizing the following cost: $K, R, T, \text{lens distortion} = argmin_{K,R,T,lens} \sum_{i=1}^{n} ||p^i - \pi(P_W^i, K, R, T)||^2$
        2. This time we also include the lens distortion (can be set to 0 for initialization).
        3. Can be minimized using Levenberg-Marquardt (more robust than Gauss-Newton to local minima).
    5. Outlier rejection
        1. There could be wrong correspondences across multi-view images -> wrong ones are called 'outliers'
        2. We should not use them for the bundle adjustment
        3. How can we reject outliers? using RANSAC
    6. RANSAC
        1. Algorithm
            1. Sample
            2. Solve
            3. Score
        2. Repeat 1-3 until the best model is found with high confidence.
3. A Few Papers Related to SFM
    1. Correspondence
        1. Universal Correspondence Network
        2. Superpoint: Self-Supervised Interest Point Detection and Description
    2. Structure and Motion Estimation
        1. Unsupervised Learning of Depth and Ego-motion from Video
        2. CNN-SLAM: Real-Time Dense Monocular SLAM
    3. Epipolar Geometry
        1. Deep Fundamental Matrix Estimation
        2. DSAC: Differentiable RANSAC for Camera Relocalization
    4. Optimization in SFM
        1. Learning to Solve Non-Linear Least Squares for Monocular Stereo
        2. BA-Net: Dense Bundle Adjustment Networks
4. 3D reconstruction project [1]
    1. Required functionalities
        1. Each group should implement a system with the following functionality:
            1. Gold standard F-matrix estimation from known correspondences. (Many of the required functions exist in CE3, and in OpenCV).
            2. Computation of an E-matrix from F and known K, and subsequent extraction of R and t from E, and visible-points constraint.

    3. Robust Perspective-n-point (PnP) estimation for adding a new view and detecting outliers.
    4. Bundle adjustment for N cameras (N≥2) using known correspondences, intrinsic camera parameters (K-matrix), and an initial guess for the set of camera poses.
    5. Use a sparsity mask for the Jacobian in the Bundle adjustment step (this speedup is absolutely essential if you use `scipy.optimize.least_squares`). Alternatively, in C++ you can use Ceres Solver, which handles the sparsity in a different way. Yet another option is to use pytorch (Ask your guide for details).
    6. Evaluate the robustness to noise in the implemented system, by measuring the camera pose errors of your reconstruction. Details can be found here.
2. In addition to the above, groups with four students should implement at least one of the following functionalities:
    1. Visualisation of the 3D model. Pick one of the following:
        1. Use Poisson surface reconstruction (or screened PSR) to obtain a volumetric representation. Try e.g. the version in Meshlab, or Kazhdan's own implementation (which has more options). Both require you to estimate colour and surface normals for the 3D-points, and save these in PLY format. Use visibility in cameras to determine normal signs. See Kazhdan and Hoppe for a theoretical description.
        2. Use space carving (as in Fitzgibbon, Cross and Zisserman) to generate a volumetric representation of your object.
        3. Use texture patches (e.g. PMVS) or billboards, to represent the texture of the estimated 3D model.
    3. Find correspondences between pairs of views, and remove false matches using epipolar geometry and cross-checking. (Allows the implemented system to use any image set with known K) Longer tracks are important for robustness, and also loop-closing correspondences (from the last view to the first in the circle) help bundle-adjustment.
    4. Use your own camera to take images of an object, and reconstruct it. This requires estimation of the K-matrix, and possibly lens distortion, followed by image rectification. Note that this also requires finding your own correspondences (functionality #2) and that you need to carefully plan how to take the images.
    5. Densify your initial sparse SfM model, using dense stereo between selected frame pairs. E.g. PatchMatch can be used, or alternatively, one of the approaches in the Multiview Stereo tutorial.
    6. Use next-best-view selection (e.g. as in Schönberger et al., or some simplified form thereof) to select the order in which cameras are added to SfM.
    7. Replace Incremental SfM with Global SfM. Use global estimation of all rotation matrices as an initialisation of Bundle Adjustment (as in Martinec and Pajdla).
    8. Write your own non-linear solver, e.g.~Levenberg-Marquardt (see the report by Madsen et al.), and use the Shur-Complement Trick to speed up the computation of the update step (see the IREG compendium).
5. Groups with five students are required to implement two of the above, and groups with six members should implement three.
6. Datasets
    1. The Visual Geometry Group at Oxford University has a number of datasets available on their website, e.g., the dinosaur sequence:

        ```
        Multiple-views datasets from the Visual Geometry Group at Oxford University
        ```

        These data sets contain coordinates of image points that are tracked between images, which means that there is both some amount of noise on the image coordinates and that these points are not visible over the entire sequence since they are occluded in some or even most of the images. The data sets contain some type of ground truth in terms of estimated camera matrices for each view.
        Notice that the dinosaur sequence is a turn-table sequence that is generated by rotating the object around a fixed axis. The acquisition geometry of this dataset makes it sensitive to the quality of the used point correspondences.
        For debugging it is useful to work with a noise-free dataset, and for this purpose, we have "cleaned" the dinosaur data set from any noise (up to the numerical accuracy of Matlab) and produced a dataset that contains 2D image points, 3D points, and the camera matrices. These are found in the BAdino2.mat file on the local ISY file system:

        ```
        /courses/TSBB15/sequences/BAdino/BAdino2.mat
        ```

        Some datasets by Carl Olsson in at LTH:

        ```
        `Carl Olsson's SfM datasets`
        ```

        The EPFL multi-view stereo datasets are also highly recommended:

        ```
        `Multi-view stereo datasets`
        ```

They all have undistorted high-res images, with ground-truth camera poses and known intrinsic camera parameters. However, you have to find the correspondences yourself here.

7. Python code
    1. Please look at the utility code for CE3. You can find it at /courses/TSBB15/python/ in the computer labs. Also, look at the Extra exercises in the lab sheet, they are meant to help you get started with the projects.
8. Matlab code
    1. In the case that you use Matlab for solving the tasks in this project, there are some software packages, that will spare you from implementing all functionalities yourself:
        1. The Visual Geometry Group at Oxford University
        2. Peter Kovesi at the University of Western Australia
        3. The LSQNONLIN optimiser in Matlab. Note that LSQNONLIN is very slow, unless a sparsity mask is used.
        4. P3P by Laurent Kneip. Note: Documentation is weird. You need to verify the input and output behaviour yourself on synthetic data (e.g. for coordinate axes). See this example for how to do this in Matlab.
9. C code
    1. You are absolutely NOT allowed to use complete SfM systems such as COLMAP, SBA, Bundler, Visual SFM, SSBA, and OpenMVG.
    2. On the other hand, you are encouraged to use a non-linear least squares package, and if you intend to make a more advanced representation of your 3D model, you may use a package for this. Some useful (and allowed) packages are listed below.
        1. Ceres Solver, a library for efficient non-linear optimization.
        2. levmar, and lmfit, Levenberg-Marquardt optimisation packages.
        3. sparseLM, a sparse Levenberg-Marquardt optimisation package (much faster, but also more complex to set up).
        4. PMVS, package to compute 3D models from images and camera poses.
        5. OpenCV, implements several local invariant features, and some geometry functions.
        6. Lambda twist P3P by Mikael Persson and Klas Nordberg.
        7. P3P by Laurent Kneip.
        8. OpenGV, a collection of geometric solvers for calibrated epipolar geometry.
10. Deliverables
    1. In order to pass, each project group should do the following:
    2. Make a design plan, and get it approved by the guide.
        1. The design plan should contain the following:
        - A list of the tasks/functionalities that will be implemented
        - A group member list, with responsibilities for each person (e.g. what task)
        - A flow-chart of the system components * A brief description of the components
    3. Deliver a good presentation at the seminar
    4. Hand in a written report to the project guide.
11. The report should contain the following:
    1. A group member list, with responsibilities for each person
    2. A description of the problem that is solved
    3. How the problem is solved
    4. What the result is (i.e. performance relative to ground truth)
    5. Why the result is what it is
    6. References to used methods
    We recommend that you use the CVPR LaTeX template when writing the report.
12. 3D reconstruction project [2]

    1. Required functionalities, 2025 version (will be updated)
        1. Each group should implement a system with the following functionality:
            1. Robust correspondence extraction using RANSAC and the epipolar constraint on a sequence of frames. (i.e. like CE1, but for many image pairs, and you may want to try different correspondences, e.g. SIFT, SuperPoint, or ROMA correspondences.)
            2. Computation of an **E**-matrix (e.g. from **F** and a known **K**, or using e.g. PoseLib) and subsequent extraction of the twisted-pair solutions to **R** and **t** from **E**, and selection of the correct one, using the visible-points constraint.
            (See LE6)

3. Robust Perspective-n-point (PnP) estimation for adding a new view and detecting outliers. Use an existing P3P solver and your own RANSAC loop.
   (See LE6,LE7)
4. Bundle adjustment for N cameras ($N \geq 2$) using known correspondences, intrinsic camera parameters (**K**-matrix), and an initial guess for the set of camera poses.
   (See LE5)
5. Improved efficiency for bundle adjustment. Options:
   - Use a sparsity mask for the Jacobian in the Bundle adjustment step (See CE1 extra task).
   - Use pytorch and autograd (See Micro Bundle Adjustment).
   - Program in C++, and use Ceres Solver, which also has automatic differentiation and a Shur complement solver. Alternatively PyCeres is also allowed. (Discuss with your guide).
6. Evaluate the robustness to noise in the implemented system, by measuring the camera pose errors of your reconstruction compared to reference poses. Details can be found here.

2. Optional functionalities
   1. In addition to the required functionalities, groups with four students should do at least one extra task from the list below. Groups of five students should to at least two extra tasks.
   2. Densification using e.g. PatchMatch, with crosschecking and the fundamental matrix constraint (See CE2)
   3. Visualisation of the 3D model. Pick one of the following:
      1. Try a radiance field method, e.g. 3D Gaussian Splatting, InstantSplat, Plenoxels or some NeRF variant, e.g. Nerf Studio. (discuss with your supervisor).
      2. Use screened Poisson surface reconstruction to obtain a volumetric representation. Try e.g. the version in Meshlab, or Kazhdan's own implementation (recommended, as it has more options). Both require you to estimate colour and surface normals for the 3D-points, and save these in PLY format. Use visibility in cameras to determine normal signs. See LE4. Theoretical details on Screened PSR can be found in Kazhdan and Hoppe.
      3. Use space carving (as in Fitzgibbon, Cross and Zisserman) to generate a volumetric representation of your object. Note that nice object silhouettes can be obtained using e.g. SAM.s
   4. Replace incremental SfM with Global SfM. Use global estimation of all rotation matrices as an initialisation of Bundle Adjustment (as in Martinec and Pajdla). Eventually Graph Attention Networks would be fun to try here, but it is probably too difficult this year (GIT repo).
   5. Make use of the points on the turntable to make camera pose estimation more accurate. This is not trivial, as the turntable is a dominant plane. You may e.g. use the fact that a view change on a plane is a homography.
   6. Add re-triangulation to your pipeline, and compare using optimal triangulation and LOST (e.g. from GTSAM) in terms of execution time and the final result quality.
   7. An idea of your own (discuss with your supervisor).

References

1. C. Barnes et al. PatchMatch: A randomized correspondence algorithm for structural image editing, ACM Transactions on Graphics (Proceedings of SIGGRAPH) 2009.
2. A. Fitzgibbon, G. Cross and A. Zisserman, Automatic 3D Model Construction for Turn-Table Sequences, in 3D Structure from Multiple Images of Large-Scale Environments, Editors Koch & Van Gool, Springer Verlag 1998, pages 155-170.
3. Y. Furukawa and C. Hernández, Multiview Stereo: A Tutorial. In Foundations and Trends in Computer Graphics and Vision Vol. 9, No. 1-2, 2013.
4. Michael Kazhdan and Hugues Hoppe, Screened Poisson Surface Reconstruction, ACM Transactions on Graphics 2013.
5. K. Madsen et al., Methods for Non-Linear Least Squares Problems, 2nd ed., DTU technical report 2004.
6. Daniel Martinec and Tomáš Pajdla, Robust Rotation and Translation Estimation in Multiview Reconstruction, CVPR 2007.
7. Klas Nordberg, Introduction to Representations and Estimation in Geometry (IREG).
8. Johannes Schönberger and Jan-Michael Frahm, Structure-from-Motion Revisited, CVPR 2016.
9. Triggs, McLauchlan, Hartley and Fitzgibbon, Bundle Adjustment - A Modern Synthesis, in Vision Algorithms - Theory and Practice, Springer Verlag, Lecture Notes in Computer Science No 1883, 2000, pages 298-372.
10. Daniel Scharstein and Richard Szeliski, A Taxonomy and Evaluation of Dense Two-Frame Stereo Correspondence Algorithms, IJCV 47(1-3), 2002
11. Brian Curless and Marc Levoy, A Volumetric Method for Building Complex Models from Range Images, SIGGRAPH'96, 1996

12. William E. Lorenzen, Harvey E. Cline, Marching cubes: A high resolution 3D surface construction algorithm, SIGGRAPH'87, 1987
13. R. Newcombe et al. KinectFusion: Real-time Dense Surface Mapping and Tracking. ISMAR'11, 2011
14. C. Loop and Z. Zhang, Computing Rectifying Homographies for Stereo Vision, ICCV 1999.

13. Evaluation metrics for Structure from Motion

    1. Each group should produce code for automatic evaluation of the estimated camera positions on the dinosaur dataset. This should be done using the following procedure:
        1. Extract the global position and orientation of cameras in the ground truth for the "my" sequence. This results in a set of K poses: (R_k, t_k).
        2. Extract the global position and orientation of your estimated cameras on the same dataset in a set of K poses: (R_k^{est}, t_k^{est})
        3. Align the two sets by estimating a similarity transformation that maps your camera positions t_k^{est} to the ground truth positions t_k, i.e. $\vec{t}_k = sM\vec{t}_k^{est} + \vec{m}$, where s is a scale change, M is the global rotation, and $\vec{m}$ is the global translation. (See e.g. the IREG compendium sec 15.2, or Horns article.)
        4. Map your estimated camera centers and orientations using $(s, M, \vec{m})$.
        5. The camera positions are mapped as: $t_k^{mapped} = sM\vec{t}_k^{est} + \vec{m}$
        6. and the orientations are mapped as: $R_k^{mapped} = R_k^{est}M^T$
        7. Comput the individual errors for the camera positions as: $e_k = \|\vec{t}_k - t_k^{mapped}\|$ and the individual angular errors for the camera orientations using the 3D angular error (the geodesic distance on SO(3)): $a_k = 2\sin^{-1}(\|R_k^{mapped} - R_k^{mapped}\|)/\sqrt{8}$. See e.g. the article by Hartley et al. for a derivation of this error measure.
        8. Summarize the errors for the entire set of poses using the mean-absolute errors (MAE) for each of the two errors above.

    References

    1. B.K.P. Horn, Closed-form solution of absolute orientation using unit quaternions, JOSA, Vol 4, 1987.
    2. Klas Nordberg, *Introduction to Representations and Estimation in Geometry* (IREG).
    3. R. Hartley, J. Trumpf, Y. Dai, and H. Li. Rotation averaging. International Journal of Computer Vision, 103(3):267-305, July 2013

References:

1. https://isy.gitlab-pages.liu.se/cvl/en/courses/TSBB15/3d-reconstruction-project/
2. https://isy.gitlab-pages.liu.se/cvl/en/courses/TSBB33/project.html
3. https://isy.gitlab-pages.liu.se/cvl/en/courses/TSBB33/evaluation.html